HPC EXAM NOTES

High Performance Computing — OpenMP Practicals

Practical 1 | Practical 2 | Practical 3

BFS/DFS · Bubble Sort / Merge Sort · Reduction Operations

Includes Theory • Algorithms • Code Explanation • Oral Q&A

OpenMP | C++ | Parallel | Graphs

Practical 1 — Parallel BFS and DFS using OpenMP

Aim:

To implement and compare sequential and parallel versions of Breadth-First Search (BFS) and Depth-First Search (DFS) graph traversal algorithms using OpenMP.

Theory:

Graph traversal is the process of visiting all nodes in a graph. Two fundamental methods are BFS and DFS. In High Performance Computing, we aim to perform these traversals faster by utilizing multiple CPU cores through OpenMP.

Concept	Description
BFS (Breadth-First Search)	Visits nodes level by level. Uses a Queue (FIFO). Explores all neighbors before going deeper.
DFS (Depth-First Search)	Goes as deep as possible before backtracking. Uses Stack or Recursion (LIFO).
Adjacency List	Graph representation: each node stores a list of its neighbors. Memory efficient for sparse graphs.
Visited Array	Boolean array tracking which nodes have been visited to prevent revisiting.

Graph used in this practical:

```
adj[0] = {1, 2} adj[1] = {0, 3, 4} adj[2] = {0, 5}
adj[3] = {1} adj[4] = {1} adj[5] = {2}

0
/ \
1 2
/ \ \
3 4 5

BFS from node 0: 0 -> 1 -> 2 -> 3 -> 4 -> 5
DFS from node 0: 0 -> 1 -> 3 -> 4 -> 2 -> 5
```

Parallel BFS — Algorithm:

- Step 1: Mark start node as visited. Push to queue.
- Step 2: While queue is not empty: pop front node, print it.
- Step 3: Use #pragma omp parallel for — multiple threads check all neighbors simultaneously.
- Step 4: Use #pragma omp critical — only ONE thread at a time marks visited and pushes to queue.
- Step 5: Double-checked locking: check visited BEFORE and INSIDE critical section for safety.

■ Why critical section? Without it, Thread A and Thread B could both check 'is node 3 visited?' simultaneously, both get 'No', and both push node 3 to the queue — causing duplicate traversal.

Parallel DFS — Algorithm:

- Step 1: #pragma omp parallel creates a team of threads.
- Step 2: #pragma omp single — only ONE thread calls parallelDFS(0) to avoid duplicate starts.
- Step 3: Visit node, print, mark visited.
- Step 4: #pragma omp parallel for — iterate over all neighbors in parallel.
- Step 5: #pragma omp task — each unvisited neighbor becomes an independent task (thread).

OpenMP Directives used in Practical 1:

Directive	Meaning	Used Where	Purpose
#pragma omp parallel for	Divide loop iterations across threads	BFS neighbor loop	Speed up neighbor checking
#pragma omp critical	Mutual exclusion block — 1 thread at a time	BFS queue/visited update	Prevent race condition
#pragma omp task	Create a new parallel task	DFS recursive call	Parallel recursion
#pragma omp single	Only one thread runs this block	DFS entry point	Prevent duplicate traversal
#pragma omp parallel	Spawn a team of threads	DFS outer block	Enable task creation

Code snippet — Parallel BFS critical section:

```
#pragma omp parallel for
for (int i = 0; i < adj[node].size(); i++) {
int neighbor = adj[node][i];
if (!visited[neighbor]) { // first check (fast)
#pragma omp critical
if (!visited[neighbor]) { // second check (safe)
visited[neighbor] = true;
q.push(neighbor);
}
}
}
```

Code snippet — Parallel DFS with tasks:

```
#pragma omp parallel for
for (int i = 0; i < adj[node].size(); i++) {
int neighbor = adj[node][i];
if (!visited[neighbor]) {
#pragma omp task
if (!visited[neighbor]) {
parallelDFS(neighbor, adj, visited);
}
}
}
```

How to compile and run:

```
g++ -fopenmp HPC1.cpp -o hpc1
./hpc1

Output:
Parallel BFS Traversal : 0 1 2 3 4 5
Parallel DFS Traversal : 0 1 3 4 2 5
```

Conclusion:

Parallel BFS and DFS using OpenMP allows graph traversal to utilize multiple CPU cores. BFS benefits from parallel neighbor-checking using critical sections to ensure thread safety. DFS benefits from OpenMP tasks which enable parallel recursive exploration. The key challenge is synchronization — preventing multiple threads from processing the same node twice.

Practical 2 — Parallel Bubble Sort and Merge Sort

Aim:

To implement and compare sequential vs parallel versions of Bubble Sort (Odd-Even Transposition) and Merge Sort using OpenMP, and measure execution time.

Theory:

Sorting algorithms arrange elements in a defined order. Bubble Sort and Merge Sort are classic comparison-based algorithms. Parallelizing sorting requires identifying independent operations that can run simultaneously without data conflicts.

Concept	Description
Bubble Sort	$O(n^2)$ — repeatedly compares and swaps adjacent elements. Simple but slow for large arrays.
Odd-Even Sort	Parallelizable variant of bubble sort. Even phases swap (1,2),(3,4)... Odd phases swap (0,1),(2,3)... Non-overlapping pairs are independent.
Merge Sort	O(n log n) — divide into halves, recursively sort, then merge. Much faster than bubble sort.
Parallel Merge Sort	Two halves sorted as independent tasks simultaneously. Uses omp task + taskwait for synchronization.
schedule(static)	Divides loop iterations into equal chunks and assigns to threads before execution.
omp taskwait	Waits for all child tasks to finish before continuing — needed before the merge step.

Bubble Sort Example:

```
Input: [5, 3, 8, 1]

Pass 1: Compare (5,3)->swap [3,5,8,1]
Compare (5,8)->ok [3,5,8,1]
Compare (8,1)->swap [3,5,1,8]
Pass 2: Compare (3,5)->ok [3,5,1,8]
Compare (5,1)->swap [3,1,5,8]
Pass 3: Compare (3,1)->swap [1,3,5,8] <- Sorted!

Time Complexity: O(n^2) | Space: O(1)
```

Why regular Bubble Sort cannot be parallelized directly:

! In regular bubble sort, each swap depends on the previous — there is a DATA DEPENDENCY between consecutive iterations. Thread A swapping positions (i, i+1) affects what Thread B will compare at position (i+1, i+2). Odd-Even Transposition solves this by ensuring the pairs being compared in a single phase do NOT overlap, making each comparison truly independent.

Odd-Even Transposition Sort — Algorithm:

- Even phase: compare and swap pairs at positions (1,2), (3,4), (5,6)... in parallel
- Odd phase: compare and swap pairs at positions (0,1), (2,3), (4,5)... in parallel
- Pairs in each phase do NOT share indices — completely independent, safe to parallelize!
- Repeat for N total phases — array is sorted after N iterations.

```
// Even phase
#pragma omp parallel for schedule(static)
for (int i = 1; i < size - 1; i += 2)
if (array[i] > array[i+1]) swap(array[i], array[i+1]);

// Odd phase
#pragma omp parallel for schedule(static)
for (int i = 0; i < size - 1; i += 2)
if (array[i] > array[i+1]) swap(array[i], array[i+1]);
```

Merge Sort Example:

```
Input: [5, 3, 8, 1]

Step 1 Divide: [5, 3] | [8, 1]
Step 2 Divide: [5] [3] | [8] [1]
Step 3 Merge: [3, 5] | [1, 8]
Step 4 Merge: [1, 3, 5, 8] <- Sorted!

Time Complexity: O(n log n) | Space: O(n)
```

Parallel Merge Sort with OpenMP tasks:

```
void p_mergesort(array, low, high, temp, depth=0) {
if (low < high) {
int mid = (low + high) / 2;

if (depth < 4) { // limit: prevents too many threads
#pragma omp task shared(array, temp)
p_mergesort(array, low, mid, temp, depth+1); // left half

#pragma omp task shared(array, temp)
p_mergesort(array, mid+1, high, temp, depth+1); // right half

#pragma omp taskwait // WAIT for both halves to finish!
} else {
// Sequential below depth 4 (overhead not worth it)
p_mergesort(array, low, mid, temp, depth+1);
p_mergesort(array, mid+1, high, temp, depth+1);
}
merge(array, low, mid, high, temp); // merge sorted halves
}
}
```

★ Depth is limited to 4 because at depth 4 we have 2^4 = 16 parallel tasks. Creating more tasks than available CPU cores gives no speedup — the overhead of task creation exceeds the benefit. Below depth 4, sequential is more efficient.

Comparison of all four sorts:

Algorithm	Complexity	Method	Performance
Sequential Bubble Sort	$O(n^2)$	Single thread, basic nested loop	Slowest
Parallel Bubble Sort	$O(n^2)$	Odd-Even phases with omp parallel for	Faster on large arrays
Sequential Merge Sort	O(n log n)	Recursive divide and conquer	Fast
Parallel Merge Sort	O(n log n)	omp task + taskwait for parallel halves	Fastest for large N

Conclusion:

Parallelizing sorting requires identifying truly independent operations. Bubble sort is parallelized using Odd-Even Transposition which eliminates data dependencies. Merge Sort is naturally parallelizable since both halves are independent — OpenMP tasks allow both halves to sort simultaneously. Parallel versions show significant speedup for large arrays (N > 10,000). For small arrays, thread overhead can make parallel slower.

Practical 3 — Parallel Reduction: Min, Max, Sum, Average

Aim:

To implement parallel reduction operations — Minimum, Maximum, Sum, and Average — on a large array using OpenMP's reduction clause, and compare performance with sequential versions.

Theory:

Reduction is a fundamental parallel computing pattern where a collection of values is combined into a single result using an associative operation. Examples include summing all elements, finding the minimum/maximum, or computing a product.

■ Without reduction, if multiple threads update the same shared variable (e.g., sum), a RACE CONDITION occurs: Thread A reads sum=10, Thread B reads sum=10, both add 5, both write sum=15. Correct answer is 20, but we get 15. OpenMP's reduction clause solves this automatically without needing a critical section.

How OpenMP Reduction works internally:

- Step 1: Each thread receives its own PRIVATE copy of the reduction variable.
- Step 2: Threads work on their assigned portion of the array independently — no conflict.
- Step 3: Each thread computes a partial result (partial sum, partial min, etc.).
- Step 4: At the end of the parallel region, all private copies are COMBINED using the specified operator.
- Step 5: Final result is stored in the original variable.

Worked Example — Parallel Sum:

```
Array: [4, 7, 2, 9, 1, 6] (N=6, 3 threads)

Thread 0 -> processes [4, 7] -> partial_sum = 11
Thread 1 -> processes [2, 9] -> partial_sum = 11
Thread 2 -> processes [1, 6] -> partial_sum = 7

Final combination: 11 + 11 + 7 = 29 (correct!)
Average = 29 / 6 = 4.83

Without reduction (race condition): could get 15, 22, or any wrong value!
```

Worked Example — Parallel Minimum:

```
Array: [4, 7, 2, 9, 1, 6] (3 threads)

Thread 0 -> min of [4, 7] = 4
Thread 1 -> min of [2, 9] = 2
Thread 2 -> min of [1, 6] = 1

Final: min(4, 2, 1) = 1 (correct!)
Initial value must be INT_MAX so any array element will be smaller.
```

All four operations — code:

```
// MINIMUM
int min_ele = INT_MAX;
#pragma omp parallel for reduction(min:min_ele)
for (int i = 0; i < n; i++)
if (array[i] < min_ele) min_ele = array[i];

// MAXIMUM
int max_ele = INT_MIN;
#pragma omp parallel for reduction(max:max_ele)
for (int i = 0; i < n; i++)
if (array[i] > max_ele) max_ele = array[i];

// SUM
long long total = 0;
#pragma omp parallel for reduction(+:total)
for (int i = 0; i < n; i++)
total += array[i];

// AVERAGE (same as sum, then divide)
total = 0;
#pragma omp parallel for reduction(+:total)
for (int i = 0; i < n; i++)
total += array[i];
double avg = (double)total / n;
```

Reduction operators supported by OpenMP:

Operator	Operation	Identity Value	Example usage
+	Sum	0	total += array[i]
*	Product	1	product *= array[i]
min	Minimum	INT_MAX	if(x < m) m = x
max	Maximum	INT_MIN	if(x > m) m = x
&&	Logical AND	1 (true)	result = result && condition
||	Logical OR	0 (false)	result = result || condition
&	Bitwise AND	~0	Bit manipulation
|	Bitwise OR	0	Bit manipulation

Time measurement using omp_get_wtime():

```
double start = omp_get_wtime(); // wall-clock time in seconds

// ... your sequential or parallel code here ...

double end = omp_get_wtime();
cout << "Time: " << (end - start) << " seconds" << endl;

// Do this for BOTH sequential and parallel to compare:
// Speedup = Sequential_Time / Parallel_Time
```

■ omp_get_wtime() measures WALL-CLOCK time (real elapsed time). Use it instead of clock() for parallel programs because clock() sums CPU time across ALL threads — a parallel program using 4 threads for 1 second would show 4 seconds!

When is parallel faster?

Array Size	Expected Performance
N < 1,000	Sequential often FASTER — thread creation overhead exceeds benefit
N = 10,000	Roughly similar performance
N > 100,000	Parallel significantly FASTER — ~2x to 4x speedup on quad-core CPU
N = 1,000,000	Parallel is much faster — speedup proportional to number of cores

Conclusion:

The OpenMP reduction clause provides a clean, efficient way to perform aggregate operations on arrays in parallel. It eliminates the need for critical sections by giving each thread a private variable, then automatically combining results. Min, Max, Sum, and Average are all classic reduction operations. For large arrays, parallel reduction offers significant speedup equal to approximately the number of available CPU cores.

Oral Exam Preparation — Questions and Answers

OpenMP Fundamentals

Q: What is OpenMP?

A: OpenMP (Open Multi-Processing) is an API for shared-memory parallel programming in C/C++ and Fortran. It uses compiler directives (#pragma omp) to parallelize code. All threads share the same memory space.

Q: What is a thread? How is it different from a process?

A: A thread is a lightweight unit of execution within a process. Threads SHARE memory (heap, global variables) but each has its own stack and registers. Processes have completely separate memory spaces. Threads are cheaper to create and communicate faster.

Q: What is a race condition? Give an example.

A: A race condition occurs when two or more threads access shared data simultaneously and the result depends on execution order. Example: Two threads read sum=10, both add 5, both write sum=15. Correct answer is 20 but we get 15.

Q: What is a critical section?

A: #pragma omp critical creates a block where only ONE thread can execute at a time. It provides mutual exclusion to prevent race conditions on shared data — but using it too much can eliminate parallelism benefits.

Q: What are shared and private variables in OpenMP?

A: SHARED variables exist once in memory, accessed by all threads simultaneously (needs sync for writes). PRIVATE variables exist as a separate copy per thread — changes don't affect others. Loop iterator variables are private by default in #pragma omp parallel for.

Q: What is omp_get_wtime()?

A: A function returning wall-clock time in seconds as a double. Subtract start from end to get elapsed time. Better than clock() for parallel programs because clock() sums CPU time across all threads, which is misleading for parallel code.

Practical 1 — BFS and DFS

Q: What data structure does BFS use? What does DFS use?

A: BFS uses a Queue (FIFO — First In First Out). Elements are processed in the order they were added. DFS uses a Stack or Recursion (LIFO — Last In First Out). Goes as deep as possible before backtracking.

Q: Why is #pragma omp critical needed in parallel BFS?

A: Multiple threads check neighbors simultaneously. Without critical, two threads could both check 'is node X visited? No' at the same time, then BOTH push X to the queue causing duplicate processing. The critical section ensures only one thread updates visited[] and pushes to queue at a time.

Q: Explain the double-checked locking pattern in BFS.

A: First check OUTSIDE the critical section (fast — most neighbors are already visited, no need to enter). Second check INSIDE the critical section (safe — ensures another thread didn't mark it visited between the first check and acquiring the lock). This reduces unnecessary critical section entry.

Q: Why is #pragma omp task used in parallel DFS?

A: DFS is recursive, so we can't directly use parallel for. Tasks allow creating work items that can run concurrently on any available thread. Each unvisited neighbor becomes an independent task, enabling parallel recursive exploration of the graph.

Practical 2 — Sorting

Q: Why can regular bubble sort NOT be parallelized directly?

A: Regular bubble sort has DATA DEPENDENCIES between consecutive iterations. Swapping positions (i, i+1) changes what needs to be compared at (i+1, i+2). Odd-Even Transposition solves this by ensuring pairs compared in each phase don't overlap — making comparisons truly independent and safe to parallelize.

Q: What is schedule(static) in OpenMP?

A: It divides loop iterations into equal-sized chunks and assigns them to threads BEFORE execution begins. Best when all iterations take equal time. Alternative: schedule(dynamic) where idle threads grab the next available chunk — better when iterations have variable execution time.

Q: What is #pragma omp taskwait?

A: A synchronization barrier that pauses execution until ALL child tasks spawned in the current scope have completed. In parallel merge sort, we MUST wait for both halves to finish sorting before we can merge them — otherwise we'd merge partially sorted data.

Q: Why limit recursion depth to 4 in parallel merge sort?

A: At depth 4 we have 2^4 = 16 simultaneous tasks. Creating tasks beyond the number of CPU cores gives no additional speedup, but the task creation overhead still costs time. Sequential execution below depth 4 is more efficient than spawning hundreds of tiny tasks.

Practical 3 — Reduction

Q: What is reduction in parallel computing?

A: Reduction combines multiple values into one using an associative operation (sum, min, max, product). OpenMP's reduction clause gives each thread a private copy of the variable, threads compute partial results independently, and at the end all copies are combined — automatically, without race conditions.

Q: What happens if you use a shared variable for sum without the reduction clause?

A: Multiple threads read and write the same variable simultaneously — a race condition. Result is unpredictable and usually wrong. You could fix it with #pragma omp critical, but then all updates would be serialized, completely eliminating the benefit of parallelism.

Q: Why initialize min_ele with INT_MAX and max_ele with INT_MIN?

A: For minimum: we start with INT_MAX (largest possible integer ~2.1 billion) so any real array element will be smaller, replacing it correctly. For maximum: INT_MIN (smallest possible integer ~-2.1 billion) ensures any element will be larger. Wrong initialization would give wrong results when array values are outside the initial value.

Q: What is Amdahl's Law? How does it relate to these practicals?

A: Amdahl's Law states that speedup is limited by the SEQUENTIAL portion of code. Formula: Speedup = 1 / (S + P/N), where S = serial fraction, P = parallel fraction, N = processors. If 20% of code must run sequentially, max speedup is 5x regardless of processors. In our practicals, merge step (P3) and queue push (P1) are sequential bottlenecks.

Quick Reference — All OpenMP Directives Used

Directive	What it does	Used in	Why
#pragma omp parallel	Creates thread team	P1 DFS, P2 MSort	Enables parallelism
#pragma omp parallel for	Parallel loop	P1 BFS, P2 Sort, P3	Distributes iterations
#pragma omp critical	One thread at a time	P1 BFS	Prevent race condition
#pragma omp task	Spawns parallel task	P1 DFS, P2 MSort	Parallel recursion
#pragma omp taskwait	Wait for tasks	P2 MSort	Synchronization
#pragma omp single	One thread executes	P1 DFS, P2 MSort	Prevent duplicates
reduction(op:var)	Thread-safe aggregate	P3 all operations	No race condition
schedule(static)	Equal work distribution	P2 Bubble Sort	Load balancing
omp_get_wtime()	Wall-clock time	P2, P3	Performance timing

Summary — Key Points to Remember

Practical	Topic	Key Concept	Key Directive
HPC-1	BFS	Queue + Critical section to prevent duplicate node processing	#pragma omp critical
HPC-1	DFS	Recursive traversal parallelized using tasks	#pragma omp task
HPC-2	Bubble Sort	Odd-Even Transposition — non-overlapping pairs are independent	schedule(static)
HPC-2	Merge Sort	Both halves of divide-step are independent tasks	#pragma omp taskwait
HPC-3	Min/Max	Each thread keeps private copy, combined at end	reduction(min/max:var)
HPC-3	Sum/Avg	Partial sums per thread, added together at end	reduction(+:total)

Remember these formulas:

```
Speedup = Sequential_Time / Parallel_Time
Efficiency = Speedup / Number_of_Processors (ideal = 1.0)
Amdahl's Law = 1 / (S + P/N) where S=serial fraction, N=processors

Compile: g++ -fopenmp filename.cpp -o output
Run: ./output
```

Good luck in your exam!

You've got this. Remember: race condition = critical section, reduction = private copies, tasks = parallel recursion.